RÉPONSES AUX QUESTIONS DU HACKER

Force brute :

Une attaque par force brute est une attaque sure pour craquer un mot de passe cependant elle peut être très couteuse en temps, voire impossible à faire pour certains cas.
Pour un seul caractère on a 94 possibilités pour notre situation où les mots de passe valide sont ceux compris entre 33 et 126 (en code ASCII).
Un mot de passe au minimum est composé de 6 caractères et au maximum 128 caractères maximum. Ce qui fait pour un processeur comme ce que j'ai processeur i7 2.8GHz (peut faire environ 2,5 milliards d'opérations par seconde) :

- 6 caractères : 94^6 ~= 7,8 * 10^6 combinaisons, ce qui peut prendre 3,1 * 10^{-3}s
- 128 caractères : 94^{128} ~= 3,6 * 10^{252} combinaisons, ce qui peut faire 2,3 * 10^{241}s (plus que l'âge de l'univers)

Force brute partielles :

Une attaque en force brute partielle consiste à tester un sous-ensemble de mots de passe, plutôt que d'essayer toutes les combinaisons possibles. Cette approche peut être plus efficace et réalisable, surtout pour des mots de passe longs ou complexes. Les sous-ensembles possibles :

Mots de Passe Courants : l'ensemble des mots de passe les plus utilisés par les utilisateurs

Mots de Passe Basés sur des Informations Personnelles : Utiliser des informations personnelles couramment utilisées dans les mots de passe, comme les noms, les dates de naissance, les noms d'animaux de compagnie, etc.

D'autres pistes :

- Utilisation des fuites de mot de passe : utiliser les fuites de mot de passe pour les utilisateurs d'autres applications dans d'autres.
- Attaques par Rainbow Tables : Utiliser des tables précalculées de hachages pour inverser les hachages de mots de passe.
- Décrypté des mots de passe hashé : Certains cryptage sont faibles et vulnérables par attaques : par collision, pré-image ou attaques arc-en-ciel.

Réflexions :

On doit gérer un fichier de type list contenant un couple de 2 string utilisateur et mot de passe, pour ce faire on peut définir un type string * string pour faciliter l'utilisation, on peut également imaginer des fonctions pour accéder au premier utilisateur et au premier mot de passe, une fonction pour accéder au deuxième couple.

On a besoin d'éliminer les doublons et d'extraire un élément par rapport à un autre. Trier la liste une fois peut réduire la complexité pour éliminer les doublons, on aura besoin de vérifier que les éléments à la suite et non parcourir toute la liste pour chaque élément, aussi pour merger deux listes sans les doublons, avec des listes triées, le merge sans doublons est de la complexité de la fonction merge.

Pour une meilleure structuration du projet, on va créer plusieurs fichier ml pour les parties du code nécessaires

Tools.ml :

On a créé une nouvelle fonction read_mdp_from_file -> string list, lit les mots de passe depuis un fichier txt en liste de string

Tri.ml :

On va créer en premier lieu des fonctions de comparaison, on aura besoin de trier le fichier selon les couples (utilisateur, mot de passe), ou selon les utilisateurs ou mots de passe, selon les besoins. On va utiliser un tri par fusion, vu sa complexité réduite O(n log n) en temps et O(n) en espace. Après la fonction split pour diviser une liste en deux, on va créer une fonction pour merger les deux liste merge_cmp qui prend en paramètre une fonction de comparaison et les deux listes à merger en les comparant selon la fonction en entré et la fonction sort qui prend la fonction de comparaison et la liste à trier.

Env.ml :

Définition du type (string * string) comme fich

Définition de fonction de manipulation des (fich list) : accès à un utilisateur, mot de passe, prochain couple.
Dans une liste déjà trier, on peut utiliser la fonction next_element qui prend en entrée une liste et

une fonction d'accès à une liste, et retourne la liste en passant les utilisateurs ou mot de passe identiques.

Pour les mots de passe, on des fonctions qui test un mot de passe et s'assure que celui-ci est valide c'est-à-dire (entre 6 et 128 caractère compris entre les caractères 33 et 126 en ASCII). Et une fonction qui hash un mot de passe en clair elle retourne un couple avec le mot de passe hashé et le mot de passe en clair.

P_fct.ml :

Dans cette partie on a les fonctions permettant d'avoir pour un élément ses nombre d'apparition, l'ensemble des doublons dans une liste, avoir une liste sans les doublons, avoir une liste des doublons en une liste où chaque élément est un utilisateur et son nombre de d apparition, qui permet de hasher l'ensemble des mot de passe en clair dont on dispose et d'avoir au final pour un fichier de fuite, les utilisateurs dont le mot de passe est connu dans les mots de passe en clair les plus populaires.

Main.ml :

Dans cette partie, on a les fonctions permettant d'avoir pour un utilisateur l'ensemble de ses mot de passe dans un fichier, pour un mot de passe l'ensemble des utilisateurs qui l'utilise dans un fichier, retourne l'ensemble des utilisateurs et leurs mot de passe en tous qui existe en double et les mots de passes avec leurs utilisateur en double, la liste des users dont le mot de passe existe dans les mots de passe clair, et une fonction qui merge deux listes sans doublons.

Utilisation :

Tu peux exécuter directement le fichier crack.exe, et tu vas avoir les réponses à tes questions, dans une petite interface faite à l'arrache où tu peux juste en choisissant un numéro de tache réaliser directement les requêtes que tu veux, tout est expliquer dessus, sinon si tu veux des détails de plus sur les utilisation des fonctions tu as besoin de savoir :

- user (prend une liste de couple) -> le premier utilisateur.
- pw (prend une liste de couple) -> le premier mot de passe.
- Next_element (prend une liste et une fonction user/pw) -> le prochain couple après le user ou pw actuel.

FONCTIONS :

check_char et check_mdp :

Utilité :

- check_char : Vérifie si tous les caractères d'un mot de passe sont dans la plage ASCII valide (33 à 126).
- check_mdp : Vérifie si un mot de passe est valide en termes de longueur (entre 6 et 128 caractères) et de caractères valides.

Manière d'utiliser :

- check_char : Fournir un mot de passe et sa longueur. Retourne true si tous les caractères sont valides, sinon false.
- check_mdp : Fournir un mot de passe. Retourne true si le mot de passe est valide, sinon false.

Temps et place mémoire nécessaires :

- Complexité temporelle :
  - check_char : O(n) où n est la longueur du mot de passe.
  - check_mdp : O(n) où n est la longueur du mot de passe.
- Complexité spatiale :
  - check_char : O(1) (utilisation constante de mémoire).
  - check_mdp : O(1) (utilisation constante de mémoire).

Sort :

Utilité : La fonction sort trie une liste en utilisant une fonction de comparaison donnée. Elle est essentielle pour organiser les données de manière ordonnée, ce qui facilite les recherches et les analyses ultérieures.

- **Manière d'utiliser :** Fournir une fonction de comparaison et une liste. La fonction retourne une liste triée selon la fonction de comparaison donnée.

Temps et place mémoire nécessaires :

- Complexité temporelle : O (n log n) où n est la longueur de la liste. Cette complexité est due à l'utilisation de l'algorithme de tri fusion (merge sort), qui divise la liste en deux sous-listes, les trie récursivement, puis les fusionne.
- Complexité spatiale : O (n), où n est la longueur de la liste. Cette complexité est due à la création de sous-listes et à la fusion des listes triées.

get_double :

Utilité : La fonction get_double retourne l'ensemble des couples qui existent en double dans une liste. Elle est utile pour identifier les doublons dans une liste de couples, ce qui peut aider à nettoyer les données ou à analyser les occurrences multiples.

Manière d'utiliser : Fournir une liste de couples. La fonction retourne une liste de couples avec leur nombre d'apparitions s'ils apparaissent plus d'une fois.

Temps et place mémoire nécessaires :

- Complexité temporelle : O (n log n), où n est la longueur de la liste. Cette complexité est due au tri de la liste et à la recherche des doublons.
- Complexité spatiale : O(n) où n est la longueur de la liste. Cette complexité est due à la création de la liste triée et à la liste des doublons.

get_element_doub_by_element :

Utilité : La fonction get_element_doub_by_element retourne un élément (utilisateur ou mot de passe) qui existe n fois avec son autre élément associé (utilisateur ou mot de passe) en liste. Elle est utile pour identifier les éléments répétés dans une liste triée de couples, en fonction de l'utilisateur ou du mot de passe recherché.

Manière d'utiliser : Fournir deux fonctions d'extraction (une pour l'utilisateur et une pour le mot de passe), une liste de couples triée, et l'élément à rechercher. La fonction retourne une liste des éléments associés à l'élément recherché.
la liste d'entrée doit etre triée par couple si on cherche des mdps pour user et par mdp si on cherche des users pour un mdp.

Temps et place mémoire nécessaires :

Complexité temporelle : O(n), où n est la longueur de la liste. Cette complexité est due à la recherche séquentielle dans la liste triée.

Complexité spatiale : O(n), où n est la longueur de la liste. Cette complexité est due à la création de la liste des éléments associés.

get_list_out_double :

Utilité : La fonction get_list_out_double retourne une liste de couples (utilisateur, mot de passe) sans doublons. Elle est utile pour nettoyer les données en éliminant les doublons, ce qui facilite les analyses et les recherches ultérieures.

Manière d'utiliser : Fournir une liste de couples. La fonction retourne une liste de couples sans doublons.

Temps et place mémoire nécessaires :

Complexité temporelle O(n), où n est la longueur de la liste. La complexité est due à la comparaison séquentielle des éléments.

Complexité spatiale : O(n), où n est la longueur de la liste. Cette complexité est due à la création de la liste triée et à la nouvelle liste sans doublons.

get_hashs :

Utilité : La fonction get_hashs retourne une liste de hachages de mots de passe valides à partir d'une liste de chaînes de caractères. Elle est utile pour transformer une liste de mots de passe en clair en une liste de hachages, en vérifiant d'abord la validité des mots de passe.

Manière d'utiliser : Fournir une liste de chaînes de caractères (mots de passe en clair). La fonction retourne une liste de hachages des mots de passe valides.

Temps et place mémoire nécessaires :

- Complexité temporelle : O(n), où n est la longueur de la liste. Cette complexité est due à la vérification de chaque mot de passe et à la génération de son hachage.
- Complexité spatiale : O(n), où n est la longueur de la liste. Cette complexité est due à la création de la nouvelle liste de hachages.

get_users_for_mdp :

Utilité : La fonction get_users_for_mdp retourne l'ensemble des utilisateurs qui existent nn fois dans une liste pour un mot de passe donné. Elle est utile pour identifier les utilisateurs qui partagent le même mot de passe, ce qui peut aider à analyser les vulnérabilités de sécurité.

Manière d'utiliser : Fournir une liste de couples (utilisateur, mot de passe) et le mot de passe à rechercher. La fonction retourne une liste d'utilisateurs associés à ce mot de passe.

Temps et place mémoire nécessaires :

- Complexité temporelle : O(n log n), où n est la longueur de la liste. Cette complexité est due au tri de la liste et à la recherche des utilisateurs.
- Complexité spatiale : O(n) où n est la longueur de la liste. Cette complexité est due à la création de la liste triée et à la liste des utilisateurs.

get_mdps_for_user :

Utilité : La fonction get_mdps_for_user retourne l'ensemble des mots de passe qui existent nn fois dans une liste pour un utilisateur donné. Elle est utile pour identifier les mots de passe associés à un utilisateur spécifique, ce qui peut aider à analyser les vulnérabilités de sécurité.

Manière d'utiliser : Fournir une liste de couples (utilisateur, mot de passe) et l'utilisateur à rechercher. La fonction retourne une liste de mots de passe associés à cet utilisateur.

Temps et place mémoire nécessaires :

- Complexité temporelle : O(n log n), où n est la longueur de la liste. Cette complexité est due au tri de la liste et à la recherche des mots de passe.
- Complexité spatiale : O(n), où n est la longueur de la liste. Cette complexité est due à la création de la liste triée et à la liste des mots de passe.

get_users_pws :

Utilité : La fonction get_users_pws retourne l'ensemble des utilisateurs qui existent nn fois dans une liste avec leurs mots de passe dans une liste de listes sans doublons. Elle est utile pour obtenir une vue consolidée des utilisateurs et de leurs mots de passe.

Manière d'utiliser : Fournir une liste de couples (utilisateur, mot de passe). La fonction retourne une liste de listes, où chaque sous-liste contient un utilisateur et ses mots de passe associés.

Temps et place mémoire nécessaires :

- Complexité temporelle : $O(n \log n)$, où n est la longueur de la liste. Cette complexité est due au tri de la liste et à la recherche des utilisateurs et de leurs mots de passe.
- Complexité spatiale : O(n) où n est la longueur de la liste. Cette complexité est due à la création de la liste triée et à la liste des utilisateurs avec leurs mots de passe.

get_users_for_identiq_mdp :

Utilité : La fonction get_users_for_identiq_mdp retourne l'ensemble des utilisateurs qui ont le même mot de passe. Elle est utile pour identifier les utilisateurs partageant le même mot de passe, ce qui peut aider à analyser les vulnérabilités de sécurité.

Manière d'utiliser : Fournir une liste de couples (utilisateur, mot de passe). La fonction retourne une liste de listes, où chaque sous-liste contient les utilisateurs ayant le même mot de passe.

Temps et place mémoire nécessaires :

- Complexité temporelle : O(n log n), où n est la longueur de la liste. Cette complexité est due au tri de la liste et à la recherche des utilisateurs ayant le même mot de passe.
- Complexité spatiale : O(n), où n est la longueur de la liste. Cette complexité est due à la création de la liste triée et à la liste des utilisateurs ayant le même mot de passe.

get_users_for_clair_mdp_main :

Utilité : La fonction get_users_for_clair_mdp_main retourne la liste des utilisateurs dont le mot de passe existe dans les mots de passe en clair. Elle est utile pour identifier les utilisateurs dont les mots de passe peuvent être vulnérables en raison de leur présence dans une liste de mots de passe courants.

Manière d'utiliser : Fournir une liste de couples (utilisateur, mot de passe). La fonction retourne une liste de listes, où chaque sous-liste contient les utilisateurs dont le mot de passe existe dans les mots de passe en clair.

Temps et place mémoire nécessaires :

- Complexité temporelle : $O(n \log n)$, où n est la longueur de la liste. Cette complexité est due au tri de la liste et à la recherche des utilisateurs dont les mots de passe existent dans les mots de passe en clair.
- Complexité spatiale : O(n), où n est la longueur de la liste. Cette complexité est due à la création de la liste triée et à la liste des utilisateurs dont les mots de passe existent dans les mots de passe en clair.

merge_out_double :

Utilité : La fonction merge_out_double fusionne deux listes de couples (utilisateur, mot de passe) en éliminant les doublons. Elle est essentielle pour combiner les informations de plusieurs fuites de données tout en évitant les redondances.

Manière d'utiliser : Fournir deux listes de couples (utilisateur, mot de passe). La fonction retourne une nouvelle liste fusionnée sans doublons.

Temps et place mémoire nécessaires :

- Complexité temporelle : $O(n \log n)$, où n est la longueur totale des deux listes. Cette complexité est due au tri des listes et à la fusion des listes triées.
- Complexité spatiale : $O(n)$, où n est la longueur totale des deux listes. Cette complexité est due à la création des listes triées et à la liste fusionnée.

Conclusion

Le projet que nous avons réalisé consiste à développer un programme en OCaml pour analyser et manipuler des données de fuites de mots de passe. Ce programme inclut plusieurs fonctions essentielles pour fusionner des listes, identifier des doublons, et extraire des informations spécifiques telles que les mots de passe associés à un utilisateur ou les utilisateurs associés à un mot de passe.

Le programme est conçu pour être efficace en termes de temps et de mémoire, avec des complexités temporelles et spatiales adaptées à la taille des données d'entrée. Les fonctions peuvent être utilisées sur n'importe quelles données respectant le format décrit.

En conclusion, ce projet fournit un ensemble complet de fonctions pour analyser et manipuler les données de fuites de mots de passe, offrant une solution robuste et flexible pour les besoins d'analyse de sécurité. Le programme est modulaire et bien structuré, ce qui facilite son utilisation et son extension.